

Unity + ROS2 기반 UR5 로봇 자동 경로 생성 마이그레이션

ROS2 HUMBLE

UNITY 2022 + URP

MOVEIT2

프로젝트 개요



대상 및 목표

대상 로봇: UR5 6-DOF 로봇 팔
(Pick & Place 시나리오)

목표: RGB 이미지, 바운딩 박스 세그멘테이션
마스크, 관절 데이터를 프레임 단위로 동기화
저장하는 합성 데이터셋 생성

기술 스택

- Unity 2022 + URP / ROS2 Humble / MoveIt2
- ScenarioManager, IK_Controller, MovePlan, UR5_Controller

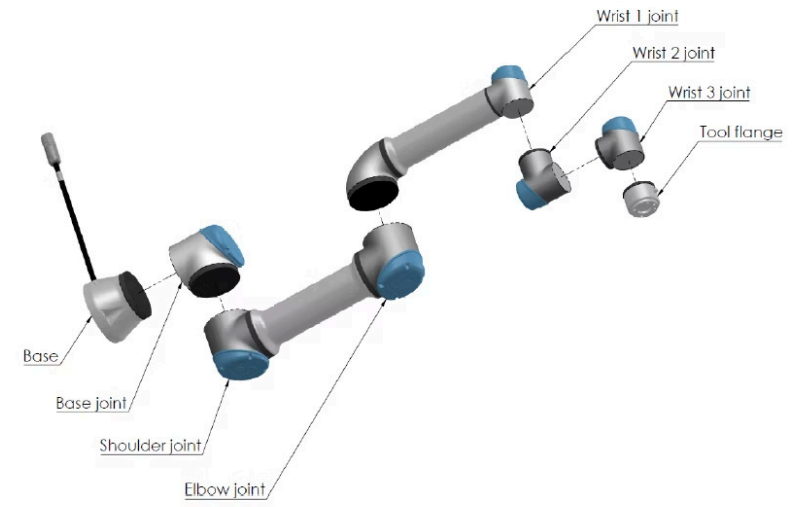


Figure 1: 로봇암의조인트, 베이스및공구플랜지.

ADAPTIVE GRIPPERS



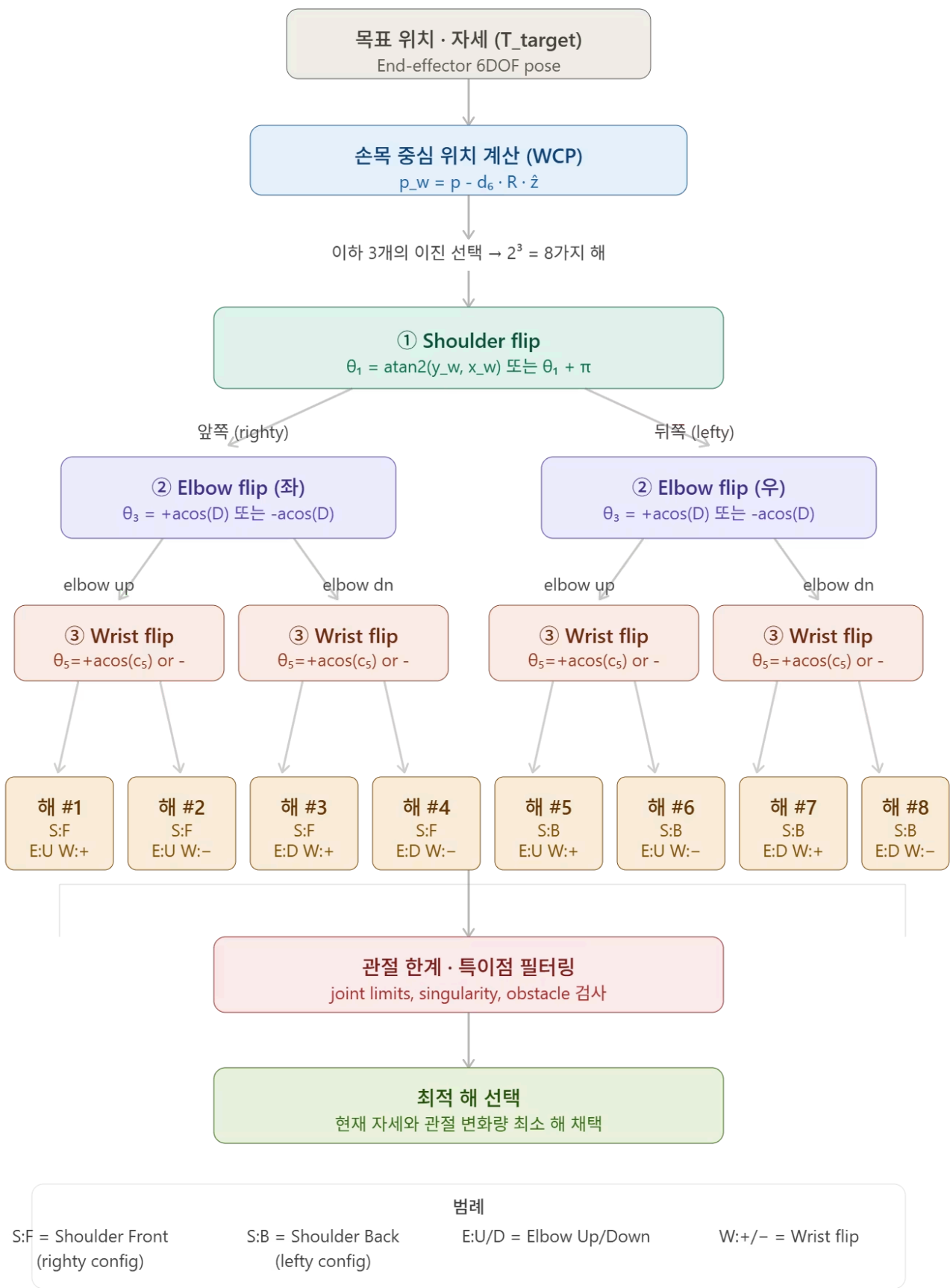
HAND-E



2F-85



2F-140



```
private Matrix4x4 hMatrix()
{
    //roll along z, pitch along y, yaw along x
    //phi = roll, theta = pitch, psi = yaw
    Matrix4x4 translate = Matrix4x4.identity;
    Matrix4x4 yaw = Matrix4x4.identity;
    Matrix4x4 pitch = Matrix4x4.identity;
    Matrix4x4 roll = Matrix4x4.identity;
    Matrix4x4 matrix;

    //set the translation matrix
    translate.m03 = x;
    translate.m13 = y;
    translate.m23 = z;

    //set roll matrix (z rotation)
    roll.m00 = (float)Math.Cos(phi);
    roll.m01 = -(float)Math.Sin(phi);
    roll.m10 = (float)Math.Sin(phi);
    roll.m11 = (float)Math.Cos(phi);

    //set pitch matrix (y rotation)
    pitch.m00 = (float)Math.Cos(theta);
    pitch.m02 = (float)Math.Sin(theta);
    pitch.m20 = -(float)Math.Sin(theta);
    pitch.m22 = (float)Math.Cos(theta);

    //set yaw matrix (x rotation)
    yaw.m11 = (float)Math.Cos(psi);
    yaw.m12 = -(float)Math.Sin(psi);
    yaw.m21 = (float)Math.Sin(psi);
    yaw.m22 = (float)Math.Cos(psi);

    // ry90
    // ur5 에셋 베이스 프레임의 90도 회전값 보정
    float y90 = Mathf.Deg2Rad * 90f;
    Matrix4x4 ry90 = Matrix4x4.identity;
    ry90.m00 = (float)Math.Cos(y90);
    ry90.m02 = (float)Math.Sin(y90);
    ry90.m20 = -(float)Math.Sin(y90);
    ry90.m22 = (float)Math.Cos(y90);

    matrix = translate * ry90 * roll * pitch * yaw;

    //Will need to set end effector rotation matrix
    /*      matrix = roll * pitch * yaw * translate;
    matrix = translate * roll * pitch * yaw* rz90;*/

    return matrix;
}
```

```
private void IK_Solver(Matrix4x4 effPos)
{
    float theta1, theta2, theta3, theta4, theta5, theta6;
    float sin1, cos1, sin5, p14Norm;
    Vector4 p05, d6, p14, p06;
    Vector2 yHat, xHat;
    Matrix4x4 efInverse, T01, T16, T65, T54, T14, T21, T32, T34;

    //***** Theta1

    d6.x = 0;
    d6.y = 0;
    d6.z = DH_d[6];
    d6.w = 1;

    p05 = effPos * d6;
    p06.x = effPos.m03;
    p06.y = effPos.m13;
    p06.z = effPos.m23;
    p06.w = effPos.m33;

    var r = Math.Sqrt(p05.x * p05.x + p05.y * p05.y);
    theta1 = (float)(Math.Atan2(p05.y, p05.x) + AcosClamped(DH_d[4] / r) + Math.PI / 2);
    //Debug.Log("theta1 = " + theta1 * Mathf.Rad2Deg);

    //fill in the first four elements of the first row of the solution matrix with theta1
    for (int i = 0; i < 4; i++)
    {
        this.solutionMatrix[0, i] = theta1;
    }

    //recalculate theta1 for the negative square root and store
    //theta1 = (float)(Math.Atan2(p05.y, p05.x) - Math.Acos(DH_d[4] / Math.Sqrt(Math.Pow(p05.x, 2) + Math.Pow(p05.y, 2))) + Math.PI / 2);
    theta1 = (float)(Math.Atan2(p05.y, p05.x) - AcosClamped(DH_d[4] / r) + Math.PI / 2);

    for (int i = 4; i < 8; i++)
    {
        this.solutionMatrix[0, i] = theta1;
    }

    //***** Theta5

    //need to take care of case when theta1 is positive sqrt
    sin1 = (float)Math.Sin(this.solutionMatrix[0, 0]);
    cos1 = (float)Math.Cos(this.solutionMatrix[0, 0]);

    //calculate positive solution
    //theta5 = (float)Math.Acos(-1 * (p06.x * sin1 - p06.y * cos1 - DH_d[4]) / DH_d[6]);
    theta5 = (float)AcosClamped(-1 * (p06.x * sin1 - p06.y * cos1 - DH_d[4]) / DH_d[6]);
    this.solutionMatrix[4, 0] = this.solutionMatrix[4, 1] = theta5;
    //Debug.Log("theta5 = " + theta5 * Mathf.Rad2Deg);

    //now store negative solution
    this.solutionMatrix[4, 2] = this.solutionMatrix[4, 3] = -theta5;

    //need to take care of case when theta1 is negative sqrt
    sin1 = (float)Math.Sin(this.solutionMatrix[0, 4]);
    cos1 = (float)Math.Cos(this.solutionMatrix[0, 4]);
    //theta5 = (float)Math.Acos(-1 * (p06.x * sin1 - p06.y * cos1 - DH_d[4]) / DH_d[6]);
    theta5 = (float)AcosClamped(-1 * (p06.x * sin1 - p06.y * cos1 - DH_d[4]) / DH_d[6]);
```


로봇 동작 플로우차트

IK_Controller.ManualMove + SceneManager.OnMoveCompleted 기반

범례 (Legend)

기본 동작 단계 (path[i])

조건부 그리퍼 회전 단계

코드 트리거 (gripper/Reset)

조건 분기 (decision)

필수 흐름

조건부 흐름 (회전 필요시)

루프 (다음 객체/시퀀스)

기호 정의

N = totalLength = paths.GetLength(0)

J6 = 6번째 조인트 (그리퍼 회전축)

코드 매핑 주석

IK_Controller.cs

- InitializePathCount():

wayToDestCount = N - 2

wayToObjCount = N - 2 - 4 = N - 6

- ManualMove(): pathNum++

→ if pathNum≥wayToObj && isOpened

→ CloseGripper

→ if pathNum≥wayToDest && isClosed

→ OpenGripper

SceneManager.cs

- OnMoveCompleted():

if objCount < movePlans.Count

→ StartNextMove()

else → ResetScenario()

- Update(): hasFinished 시

sequenceCount++ 후 다음 시퀀스

MovePlan.cs / Scenario9MovePlan.cs

- 표준 N=10 경로 구성:

[0]작업준비 [1]1차이동

[2]그리퍼회전 [3]파지점접근

[4]파지유지 [5]후퇴

[6]2차이동 [7]목표점접근

[8]방출유지 [9]최종후퇴

- N=9, N=5 경로는 회전 단계 생략

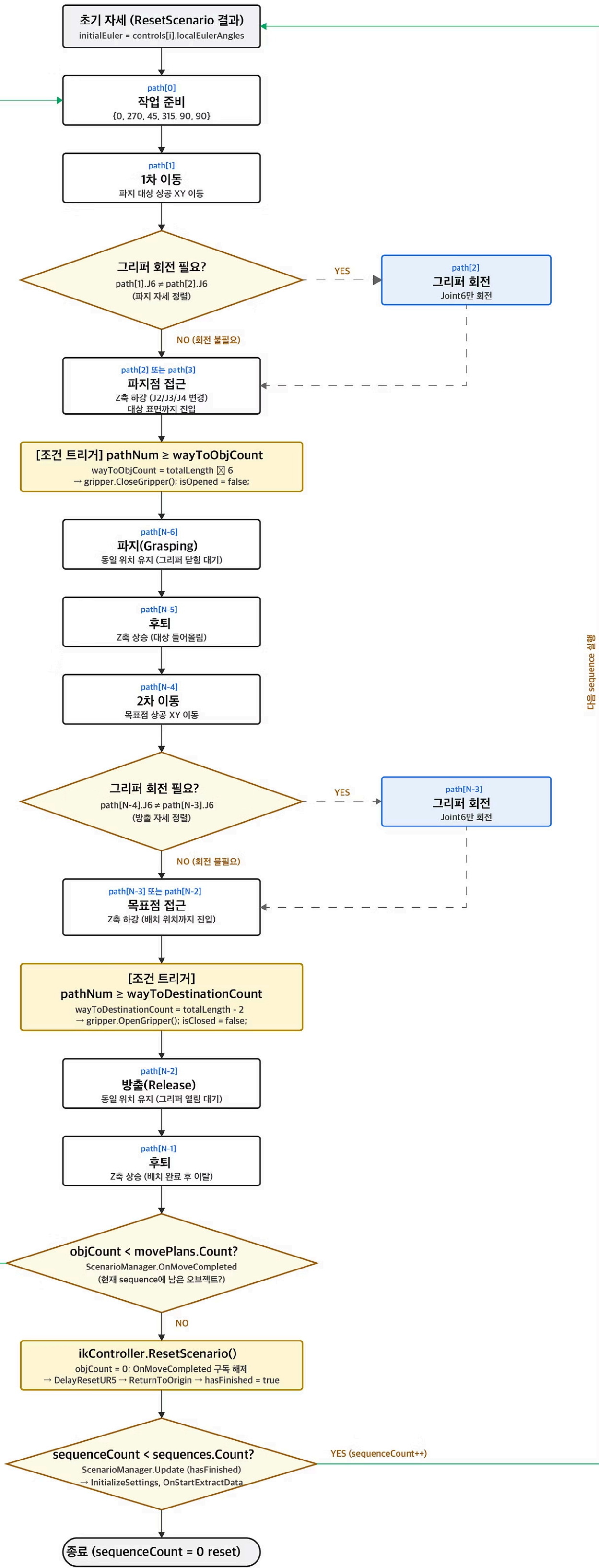
(joint6 변화량이 작거나 1차이동에

회전이 통합된 경우)

* backPathCount=4 하드코딩

→ 짧은 경로(N<6)에서 트리거 오작동 위험

다음 movePlan (objCount++)



다음 sequence 실행

수동 경로 생성 방식

① 수동 조작

Lab 씬에서 개발자가 로봇을 직접 조작 → PathGenerator.cs로 관절값 콘솔 로그 추출 → 코드에 복붙

② 하드코딩

MovePlan.cs에 물체별 사전 계산된 관절 각도 배열 float[,] anglePathes를 직접 정의

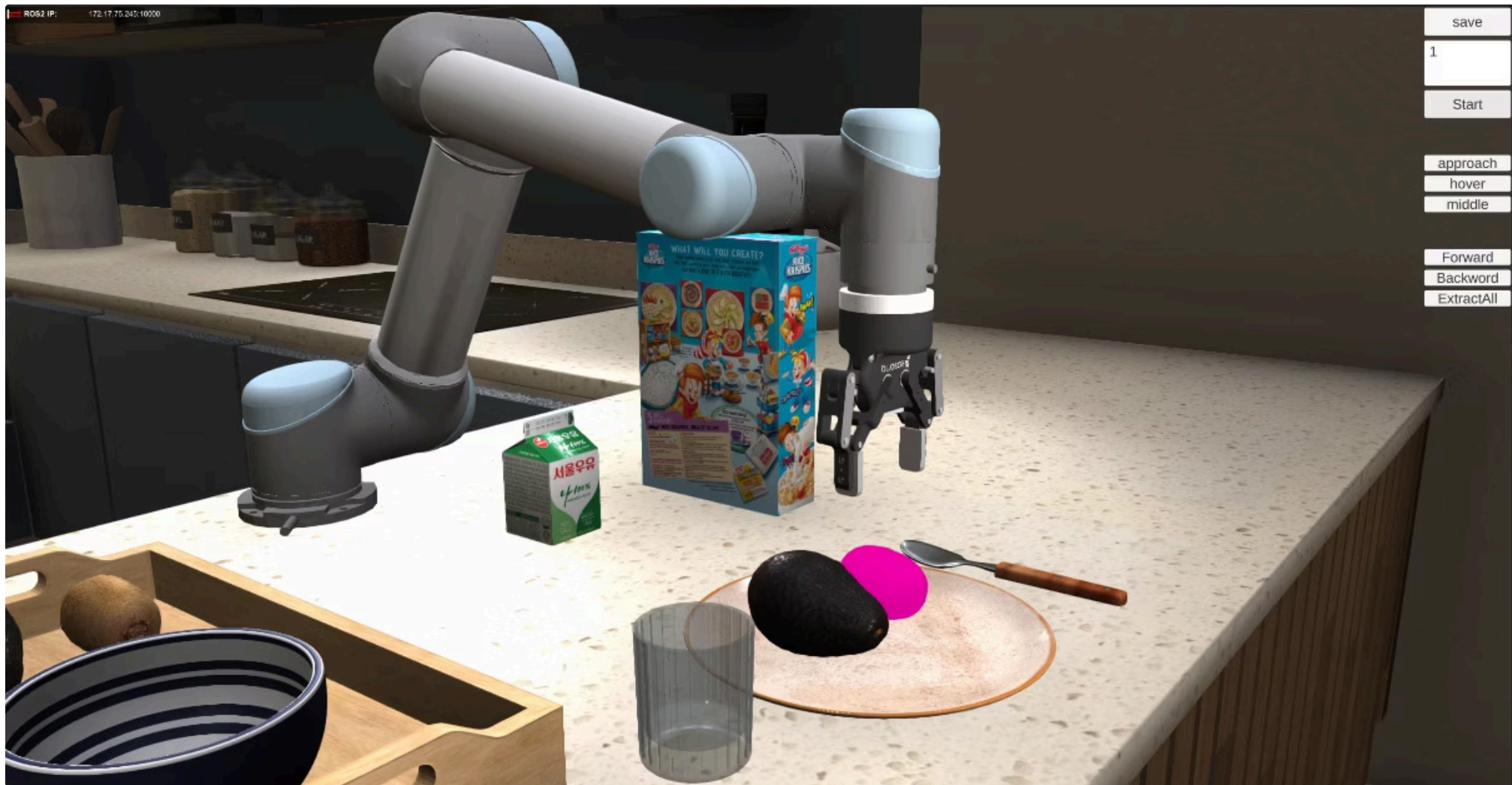
```
public MovePlan(int num, List<GameObject> grabbableObjs, Transform parent)
{
    //if (grabbableObjs[num] == null) return;
    this.currentObj = grabbableObjs[num];
    this.currentObjNum = num;
    this.currentObjName = grabbableObjs[num].name;
    this.currentPos = SetTargetPosition(grabbableObjs[num]);
    this.curParent = parent;
    this.anglePathes = SelectPathsByObjectNum(num);
}
```

③ 선형 보간 재생

IK_Controller.StartScenario() → ManualMove() 코루틴으로 6관절 선형 보간 재생

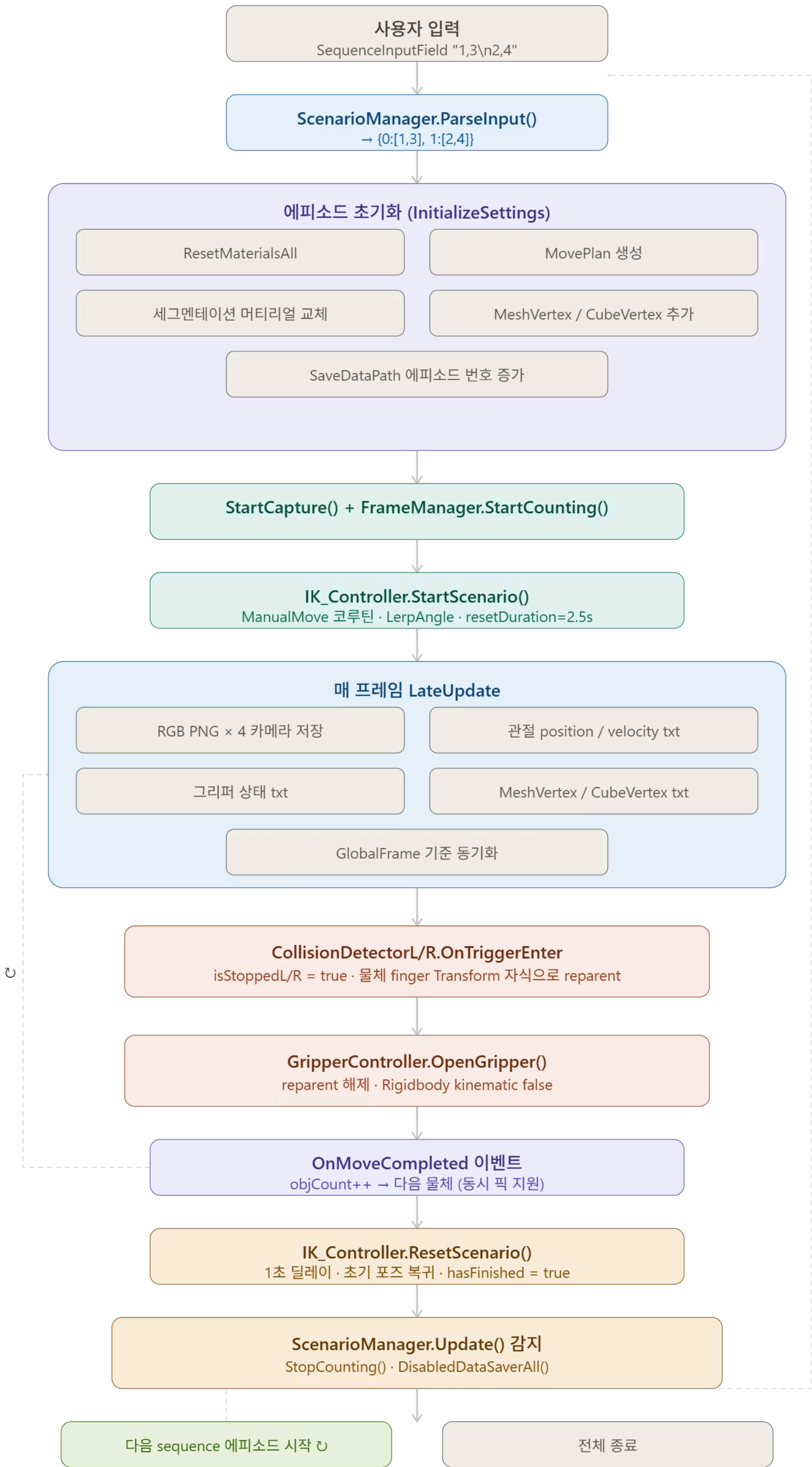
④ 전량 수동 등록

물체 7종 × 경유점 5~10개를 모두 수동으로 등록 — 신규 물체 추가 시 수 시간 소요

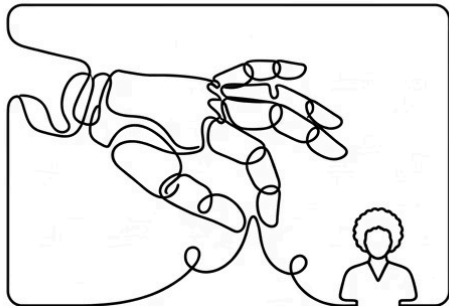


grabbables1 (cereal, 10 waypoints):

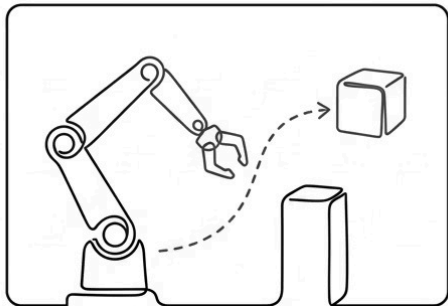
idx	[J1°]	[J2°]	[J3°]	[J4°]	[J5°]	[J6°]	역할
0	0.0	270.0	45.0	315.0	90.0	90.0	홈 자세
1	330.34	268.61	76.67	284.72	90.0	90.0	물체 상공 접근
2	330.34	268.61	76.67	284.72	90.0	119.66	gripper 방향 보정 ← J6 변화
3	330.34	266.89	94.55	268.56	90.0	119.66	물체 높이로 하강
4	330.34	266.89	94.55	268.56	90.0	119.66	그립 (2프레임 대기)
5	330.34	273.07	58.94	298.00	90.0	119.66	물체 들고 상승 ← wayToObjCount CloseGripper() 호출
6	56.33	284.60	45.21	300.19	90.0	123.67	목적지 방향 이동
7	56.33	277.53	77.03	275.44	90.0	123.67	목적지 하강
8	56.33	277.53	77.03	275.44	90.0	123.67	내려놓기 대기
9	56.33	279.40	64.43	286.17	90.0	123.67	← wayToDestinationCount OpenGripper() 호출



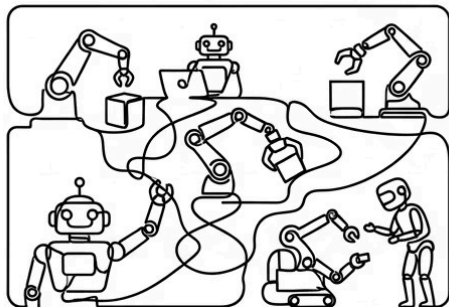
수동 방식의 한계



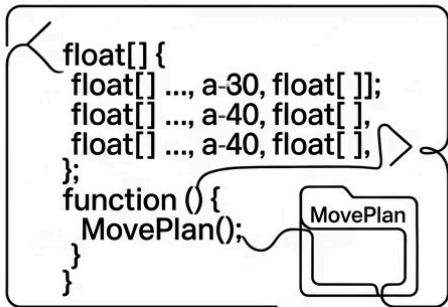
**개발자 숙련도 의존
(연산자 의존성)**



**환경 변화 적응 불가
(객체 이동, 장애물)**



**비 확장성 (시나리오
(시나리오 확장에따른 업무량 증가))**



**코드와 데이터 혼재
(MovePlan 데이터 컨테이너)**

수동 방식은 초기 프로토타입에는 유효하지만, 시나리오가 확장될수록 유지보수 비용이 기하급수적으로 증가합니다.

❌ 물체 위치가 조금만 바뀌어도 수동 재튜닝

❌ 새 물체 추가 = 새 float[N,6] 배열 수동 작성

❌ 충돌 없다는 보장이 없음 (관절 한계 밖으로 갈 수 있음)

⚠️ **로봇 공학 관련 도메인 지식**

IK 연산이 왜 물리적으로 확정된 값을 내는지, 그리퍼가 물체를 집어야 할 때와 놓아야 할 때의 충돌 판정 조건이 어떤 원리로 작동하는지를 먼저 이해

ROS2 도입

변경 전

1

SceneManager

2

MovePlan

(float 배열 꺼냄)

3

IK_Controller

(관절 직접 움직임)

변경 후

SceneManager ← useROS2 필드 추가됨

- [useROS2=false] 기존 그대로
- [useROS2=true]
 - **GoalPosePublisher** ← 새로 추가 (신호 보내는 역할)
 - ROS2로 호출 전송
 - (ROS2가 경로 계산 후 반환)
 - **JointTrajectorySubscriber** ← 새로 추가 (신호 받는 역할)
 - IK_Controller.StartScenario() - 기존 그대로

핵심 설계 포인트 — 데이터 출처만 바꾼 것

WSL2 ROS2 쪽은?

ROS2 패키지(ur5_moveit_bridge)를 활용할 경로 데이터에 대한 중간 번역기 역할로 도입

JointStatePublisher (10Hz 상시 발행)
/joint_states
→ robot_state_publisher → TF

moveit_bridge_node

Unity /goal_pose

- ✓ 목표 지점 위치 (xyz)
- ✓ 목표 자세 (quaternion)

Movelt2 경로 계산

Unity /joint_trajectory

Unity → IK_Controller 실행

변경 사항 : Unity 스크립트 3개 신규 + 1개 필드 추가, ROS2 브릿지 노드 1개.

변경 범위 최소화

데이터 흐름 비교

1

기존 방식:

MovePlan.anglePathes (float 배열) → IK_Controller

↑ 미리 녹화된 각도

2

신규 방식:

ROS2 JointTrajectory (float 배열) → IK_Controller

↑ ROS2가 계산한 각도

변경 사항

1

변경 대상 — 2개 파일만

MovePlan.cs: float[,] 배열 제거 → Pick/Place Pose 필드 + PathRequestService (ROS2 요청/캐싱)

IK_Controller.cs: StartScenario(float[,]) → StartScenario(JointTrajectory) 인터페이스 교체

2

변경 없는 레이어 — 완전 유지

DATA, Recording, ScenarioManager 레이어는 수정 없이 그대로 유지. 하위 호환성 보장으로 기존 데이터 파이프라인에 영향 없음.

☑️ 최소 침습적(Minimally Invasive) 마이그레이션 — 기존 아키텍처를 보존하면서 경로 생성 레이어만 교체

IK_Controller에서 활용되는 경로 데이터가 변경되어도 인터페이스(StartScenario)가 같기 때문에 동일한 로직으로 동작함.

ROS2 MoveIt2 — 경로 계획의 패러다임 전환

MoveIt2 = 내비게이션 앱

목적지 좌표만 입력하면 알아서 경로를 계산합니다.

어디든
갈 수 있음

장애물 회피 메커니즘
적용

계산 시간 소요
(5초 이내)

매번 동일경로 추출X

GoalPosePublisher → /goal_pose → MoveIt2 계산 → /joint_trajectory 반환

“그리퍼를 이 위치/방향으로 가져다줘”만 말하면 경로는 알아서 계산됩니다.

- ☑ | 현재 방식은 '어떻게 움직여라'를 직접 지정,
| ROS2는 '어디로 가라'만 지정하면 방법은 알아서.

Before vs After 비교

항목	● 수동 방식 (Before)	✓ ROS2 방식 (After)
신규 물체 추가	수동 조작 수 시간	포즈 지정 수 분
충돌 회피	없음	Movelt2 OMPL 자동 처리
위치 변동 대응	전체 경로 재수집	좌표만 수정
경로 품질	개발자 숙련도 의존	플래너 최적화로 균일
시나리오 확장성	7종 한정	임의 물체·배치 즉시 확장

2개

수정 파일

전체 코드베이스 중 단 2개 파일만 변경

7종→∞

시나리오 확장

고정 7종에서 임의 물체·배치로 무한 확장

0

파이프라인 영향

기존 데이터 파이프라인 변경 없음

결론 및 시사점

→ 최소 침습적 마이그레이션

기존 이벤트 기반 아키텍처를 유지하면서 경로 생성 레이어만 교체 — 리스크 최소화과 생산성 향상을 동시에 달성

→ 수동 하드코딩의 명확한 한계

초기 빠른 구현에는 유효하지만, 확장성과 유지보수성에서 한계가 명확히 드러남

→ 환경 변화 적응형 경로 계획 실현

ROS2 MoveIt2 연동으로 물체 위치나 장애물 변화에 실시간으로 대응하는 경로 계획 가능

→ 범용 적용 가능한 브릿지 패턴

Unity 시뮬레이션과 ROS2 생태계를 연결하는 이 패턴은 다양한 로봇 학습 데이터 생성 프로젝트에 즉시 적용 가능

→ 도메인 지식과 객체 상호작용 이해의 필수성

자동화를 완성하기 위해서는, 각 컴포넌트 간 상호작용(IK, 그리퍼 물리, 충돌 판정 등)과 로봇 공학 기초 물리 법칙을 직접 이해하는 과정의 선행이 요, 도메인 지식 없이는 ROS2 자동화도 블랙박스에 불과

